

Portfolio Framed Cases

Voice Card

Clear, honest, practical, warm, no buzzwords.

Bio

I am Harshini Priya, a backend AI engineering learner building practical APIs, data pipelines, and small AI-backed systems. I like projects where the result is easy to check: a working endpoint, a clean JSON file, a database row that survives restart, or a report someone can open.

Contact / CTA

See the code on GitHub:

https://github.com/Harshini-priya989/FlyRank_W1_A3_Containerized_Postgres

I am looking for backend or AI engineering opportunities where I can keep building reliable, useful systems from real requirements.

Case Study 1: Containerized Task API

The Problem

My first task API worked, but it needed to become closer to a real backend. A database running only as a local file is useful for learning, but production-style work needs a database server and a repeatable way to start the whole stack. The challenge was to keep the API behavior the same while moving the storage to PostgreSQL in Docker.

What I Did

I built a FastAPI task API backed by PostgreSQL. I added a `Dockerfile`, `compose.yaml`, `.env.example`, and a named Docker volume so the API and database can start with one command. I kept the database logic in `repository.py`, created the `tasks` table automatically, seeded three tasks only when the table was empty, and used parameterized psycopg queries instead of building SQL strings from user input.

The main decision was to keep the route layer simple. The endpoints did not need to know whether the data lived in memory, SQLite, or PostgreSQL. Only the repository layer changed.

What Came Of It

The stack runs with `docker compose up --build`. The task rows are visible both through the API and directly in PostgreSQL, and the named volume keeps data after `docker compose down` and restart. This gave me a practical version of a backend setup that another person can clone and run.

Case Study 2: Supabase Auth API

The Problem

The task API was open to everyone. That is fine for a small demo, but not for a real system where some routes should only answer for logged-in users. I needed to add signup, login, logout, and protected routes without storing passwords myself.

What I Did

I added Supabase Auth to the FastAPI project. The app now has `/auth/signup``, `/auth/login``, `/auth/logout``, `/public/info``, `/protected/profile``, and `/protected/dashboard``. I used environment variables for the Supabase URL and anon key, kept `.env`` ignored, and documented the required values in `.env.example``.

For protected routes, I created a reusable bearer-token dependency. It reads ``Authorization: Bearer <token>``, verifies the token with Supabase, and rejects missing or invalid tokens with ``401``. I also configured FastAPI's ``HTTPBearer`` scheme so Swagger shows protected-route padlocks.

What Came Of It

The API now has a real authentication boundary. Public routes remain open, protected routes require a valid token, and the auth logic is reusable instead of copied into every route. The project is ready for live Supabase testing once personal Supabase credentials are placed in `.env``.

Case Study 3: Polite Book Scraper

The Problem

Scraping can easily become messy or rude: repeated requests, unclear source rules, broken pages crashing the run, and unvalidated data slipping into output. I needed to collect data from a practice sandbox in a way that was polite, repeatable, and honest about failures.

What I Did

I built a Python scraper for Books to Scrape. It checks the target, documents the scope, sends an identifying user-agent, uses a timeout, waits between real requests, and caches HTML locally. It discovers the first three catalogue pages from the site's own next links and finds 60 unique book URLs.

For each book page, the scraper extracts title, product URL, price text, availability text, rating text, description, source page, and fetch timestamp. It normalizes price into ``price_gbp``, converts rating text into a number, validates every record with Pydantic, and writes only valid records to ``output/books.json``. A deliberately broken URL is logged to ``errors.json`` without stopping the run.

What Came Of It

The scraper produced 60 valid unique book records and a ``run-report.json`` showing the run details. A rerun used cache hits and still produced 60 records, not duplicates. The report also recorded one failed page from the deliberate bad URL, which proved the scraper could finish useful work even when one page failed.

Case Study 4: Book Insights API Capstone

The Problem

The scraper created useful book data, but a JSON file by itself is not a product. A person still has to open it, search manually, compare prices, guess categories, and prepare a report. I wanted to turn the collected data into a small backend tool that makes those steps much faster.

What I Did

I built Book Insights API as my 10x Solution capstone. It imports the W5 book dataset into SQLite and exposes endpoints for listing books, searching, recommendations, AI-style tagging, background imports, usage logs, and PDF report generation. I added a demo login, bearer-token protection for sensitive routes, a 60-second cache for repeated searches, and a PDF report endpoint.

The capstone implements these concepts from the program: API endpoints, database persistence, authentication, background jobs, PDF reporting, caching, and LLM-style tagging with a cost log. I chose a small scope on purpose: no polished frontend, no real user base, and no paid services.

What Came Of It

The verified demo flow starts the API, loads 60 books, searches the dataset, logs in with a demo user, runs a protected tagging endpoint, generates a PDF report, and downloads the latest report. The result is a backend project that connects several internship concepts into one working system.

Before / After

Generic AI Line

I developed a results-driven backend solution leveraging cutting-edge technologies to optimize data workflows and improve user outcomes.

Edited Version

I built a small backend that turns scraped book data into searchable records, protected endpoints, cached recommendations, and a PDF report someone can open.